

Alamein International University
Faculty of Computer Science & Engineering

SecureSight : Smart Surveillance System to Detect Unusual Activities in Real Time

A project submitted
in partial fulfillment of the requirements for the BSc. degree

Presented by:
Ahmed Mostafa
Mohamed Hassona
Abdullah Abdalgawad
Omnia Adel
Mariam Elrafei
Khalid Abdelsalam
Tasneem Ibrahim

Supervised by
Dr Essam Abdellatef

January 2026

ACKNOWLEDGEMENT

This project was completed through the combined efforts of multiple contributors working toward a common academic objective. The development of SecureSight involved coordinated work across data preparation, model training and evaluation, system design, and application integration. Each stage of the project benefited from collaboration, discussion, and shared technical exploration.

The project made extensive use of established benchmark resources, most notably the UCF Crime Dataset and its official temporal annotations. These resources enabled structured experimentation, reproducible results, and meaningful evaluation of intelligent video anomaly detection techniques within a realistic surveillance context.

The work also benefited from academic guidance and ongoing feedback throughout the development process. This guidance supported adherence to software engineering practices, ethical considerations, and methodological rigor.

Finally, acknowledgement is given to the broader research and open source community whose prior work in deep learning, video understanding, and anomaly detection provided essential foundations for this project.

DECLARATION

This is to declare that the project entitled “SecureSight” is an original work done by undersigned, in partial fulfillment of the requirements for the BSc. degree at the Faculty of Computer Science & Engineering (CSE), Alamein International University (AIU).

All the analysis, design and system development have been accomplished by the undersigned. Moreover, this project has not been submitted to any other college or university.

Ahmed Mostafa

Mohamed Hassona

Abdullah Abdalgawad

Omnia Adel

Mariam Elrafei

Khalid Abdelsalam

Tasneem Ibrahim

ABSTRACT

SecureSight is an intelligent video anomaly detection system designed for smart surveillance applications. The system analyzes live video streams, as well as uploaded videos for testing purposes, and generates alerts when abnormal or suspicious activities are detected. SecureSight integrates a deep learning-based anomaly detection model trained on the UCF Crime Dataset, which includes multiple crime categories in addition to normal behavior. Official temporal annotations were used during training, allowing the model to learn from accurately labeled abnormal time segments and improving its performance compared to earlier experiments based on manually extracted features.

The anomaly detection engine is based on a Vision Transformer architecture, chosen for its ability to model long-range temporal relationships across video sequences. Initial experiments achieved limited accuracy, ranging between 50% and 60%. After incorporating temporal annotations, the model performance improved to approximately 70% accuracy. Additional experiments were conducted using evolutionary optimization techniques, including Grey Wolf Optimization, Particle Swarm Optimization, Spider Monkey Optimization, Shuffled Frog Leaping Algorithm, and Harris Hawks Optimization. However, these methods did not outperform the baseline Vision Transformer model, which was therefore selected for deployment.

SecureSight also provides a web-based interface that allows users to submit video input and view anomaly classification results. To improve safety and reliability, inference-time safeguards were added, including confidence thresholding and temporal confirmation delays to reduce false alerts. Privacy considerations were addressed by processing videos without permanent storage. Overall, SecureSight presents a complete software system that combines deep learning, real-time video analysis, ethical safeguards, and a user-friendly interface for intelligent video anomaly detection in smart surveillance environments.

Table of Content

Chapter 1: Introduction and Background 8
1.1 Overview 8
1.2 Motivation 9
1.3 Challenges 10
1.4 Problem Statement 12
1.5 Objectives 13
1.6 Limitations of Existing Solutions 14
1.7 Scope of the Project 14
Chapter 2: Related Technologies 15
2.1 Video Anomaly Detection Frameworks 15
2.2 Deep Spatiotemporal Feature Extraction Backbones 16
2.3 Vision Transformers for Sequence Modeling 16
Chapter 3: System Requirements and Analysis 17
3.1 Functional Requirements 17
3.2 Non-Functional Requirements 19
3.3 System Analysis & Modeling Diagrams 19
3.3.1 Behavioral Boundaries: Use Case Analysis 20
3.3.2 System Data-Interchange Flow: Level 0 Context Diagram 22
3.3.3 Decoupled Sub-Service Landscape: Level 1 Component Diagram 23
3.4 System Static Structure: Class Diagram 24
3.5 Constraints and Risk Analysis 25
3.5.1 System Constraints 25
3.5.2 Risk Analysis and Mitigation Matrix 25
3.6 Planning and Management 26
3.7 Work Breakdown Structure (WBS) and Milestones 28
3.8 Team Roles and Responsibilities 29
3.9 Tools, Technologies, and Resources 30
Chapter 4: System Design and Architecture 31
4.1 System Block Diagram 31
4.1.1 Data Ingestion and Frame Preprocessing 32
4.1.2 Dual-Branch Feature Extraction 32
4.1.3 Tensor Fusion and Sequence Modeling 33
4.1.4 Decision Engine and Safety Thresholds 33

4.2 Deep Learning Model 33
4.2.1 Parallel Feature Extraction and Structural Factorization 33
4.2.2 Tensor Fusion and Projection Matrix 34
4.2.3 Sequence Modeling via a Temporal Vision Transformer (ViT) 35
4.3 Dataset Engineering & Imbalance Mitigation 36
4.3.1 Samples from dataset: 39
Chapter 5: Implementation Details 40
5.1 Web System Architecture & Integrated Data Flow 40
5.2 Model Serving Pipelines and Framework Setup 41
5.3 Backend API Gateway and Endpoint Logic 42
5.4 Inference-Time Safeguards and Decision Engine 42
5.5 System Integration and Workflow 43
5.6 Model Training and Validation Strategy 43
5.7 Real-Time Processing and Optimization 44
5.8 Error Handling and System Robustness 45
5.9 Communication Protocols and Data Management 46
5.10 Deployment and Testing Environment 47
5.11 User Interface Design & Alerts 47
Chapter 6: Ethical, Privacy, and Regulatory Considerations 50
6.1 Data Privacy Framework and Protection Principles 50
6.1.1 The Ephemeral Processing Paradigm 50
6.1.2 Conditional Event Logging 51
6.2 Algorithmic Fairness and Bias Mitigation 52
References 53

Chapter 1: Introduction and Background

1.1 Overview

Surveillance camera networks have become a ubiquitous and foundational component of modern urban infrastructure. Globally, the footprint of public and private security cameras has expanded drastically, with estimates indicating that the global installed base of closed-circuit television (CCTV) units exceeded one billion units by 2025. These networks continuously monitor critical municipal spaces, commercial sectors, and transport hubs to maintain public safety. However, a severe systemic gap exists within this sprawling infrastructure: the vast majority of the video footage generated by these networks remains entirely unreviewed and analytically unutilized. Traditional systems act purely as passive storage vaults, capturing events as they happen but offering zero immediate contextual understanding.

To address this gap, this thesis presents **SecureSight**, an intelligent, automated, real-time video surveillance framework. SecureSight transforms the legacy passive recording paradigm into an active, real-time detection environment by automatically identifying multiple classes of anomalous behaviors and routing targeted, role-differentiated alerts to the appropriate emergency response authorities—such as police, hospitals, and firefighting services.

The primary technical core of SecureSight consists of a multi-modal, two-stream deep learning architecture integrated with a temporal reasoning layer:

- **Spatial Feature Extraction Branch:** Utilizes a pre-trained ResNet-50 convolutional neural network to process single, representative video frames, capturing high-resolution, static visual appearance features and semantic scene contexts.
- **Temporal Feature Extraction Branch:** Utilizes a pre-trained R(2+1)D network to analyze consecutive 16-frame clip volumes, extracting complex, local human motion dynamics and short-range spatiotemporal patterns.
- **Sequence Modeling Module:** Employs a compact Vision Transformer (ViT) encoder that processes a sequential sliding

window of fused spatiotemporal clip representations. By leveraging multi-head self-attention mechanisms, the ViT models long-range temporal context and relational dependencies across non-adjacent segments.

The complete pipeline is rigorously trained and benchmarked on the highly challenging, real-world UCF-Crime dataset using precise, clip-level temporal annotations. Beyond the core model architecture, SecureSight bridges deep learning theory and software engineering through a production-ready systems architecture. The framework embeds the inference pipeline into a modular server environment, prevents data loss under network failure via localized offline buffering mechanisms, and displays critical alert feeds across dedicated, stakeholder-specific subscriber dashboards.

1.2 Motivation

The core motivation driving this research stems from a profound paradox inherent to modern urban surveillance: cities around the world possess an unprecedented visual capacity to capture information, yet they lack the simultaneous processing capability to comprehend it in real time. Legacy video monitoring frameworks rely fundamentally on human operators to observe live terminal feeds. This dependency introduces significant operational friction due to well-documented psychological and cognitive constraints:

- **Vigilance Degradation and Cognitive Fatigue:** Human attentional limits break down rapidly during sustained monitoring tasks. Empirical studies on human operators show that target detection performance degrades substantially within only thirty minutes of continuous watch.
- **Scale Overload:** With modern municipal deployments scaling into thousands of cameras per district, it is mathematically and economically impossible for human teams to audit more than a tiny fraction of incoming streams simultaneously.
- **Delayed, Reactive Responding:** Because operators miss unfolding threats due to limited attention, security departments are forced into a post-incident reactive posture. Footage is generally retrieved from local edge archives only *after* a crime, accident, or

hazard has taken place, completely missing the opportunity to intervene, mitigate damage, or save lives in real time.

This operational failure has severe social implications, directly contributing to elevated anxiety among urban citizens, with surveys indicating that up to 72% of locals feel unsafe in crowded public areas.

Recent advancements in computer vision and deep learning offer a powerful path toward resolving this issue. Over the past decade, convolutional neural networks (CNNs) have mastered the extraction of granular, localized visual attributes, while attention-based Transformer models have proven highly effective at capturing macro-level sequence structures across extensive temporal horizons.

The technical motivation of this thesis is to deliberately synthesize these distinct paradigms into a unified, lightweight framework. By merging frame-level appearance data with clip-level motion dynamics and modeling their long-term sequence interactions, a surveillance system can shift from a passive, historical recording mechanism into a proactive, intelligent agent capable of immediate crisis understanding and rapid operational orchestration.

1.3 Challenges

Designing and executing an operational, real-time video anomaly detection system introduces a matrix of complex academic and engineering challenges across data, modeling, and physical deployment layers:

- **Extreme Real-World Environmental Variance:** Unlike controlled laboratory datasets, authentic surveillance footage captured in the wild features extreme variations in camera viewpoints, low geometric resolutions, variable frame rates, noisy backgrounds, and unpredictable illumination shifts across day, night, and weather cycles.
- **Fine-Grained Dual-Modality Visual Representation:** Anomalies in video are highly multi-faceted. Certain abnormal behaviors are characterized purely by static structural changes (e.g., a person wielding a weapon or an unattended bag left on a walkway) ,

whereas others are defined entirely by explosive kinematic actions (e.g., fighting, shunning, or high-velocity vehicle collisions). Capturing both orthogonal streams of data efficiently without multiplying processing pipelines is a delicate architectural challenge.

- **Severe Class Imbalance and Dataset Representation:** Authentic data collections are intrinsically skewed. In the real world, "Normal" visual behavior constitutes more than 99% of total street activity, while critical anomalies are sparse, brief, and heterogeneous. In the training pool of the UCF-Crime dataset, the Normal class accounts for nearly half of the dataset by total frame volume. Training a deep network under these conditions often induces a severe prediction bias toward normality, hiding true anomalies and elevating false-negative rates.

- **Label Noise and Supervision Paradigms:** The broader literature frequently relies on weakly supervised multiple-instance learning (MIL), where labels are provided only at a coarse, video-wide level. This introduces massive label noise, as an explicit anomaly lasting only 5 seconds within a 10-minute video forces the network to calculate alignments across massive volumes of normal frames during optimization. Shifting to precise, clip-level temporal supervision eliminates this noise but demands rigorous data partitioning, structural restructuring, and highly sensitive optimization tracking.

- **The Latency-Accuracy Trade-Off at the Edge:** In production environments, high model accuracy is useless if achieved via massive backbones that introduce multi-second processing latencies. Conversely, ultra-lightweight models that achieve high frame rates but flood municipal networks with false-positive alarms trigger alert fatigue, destroying operator trust and wasting public emergency resources. Balancing confidence screening and temporal consistency verification against sub-second processing throughput bounds remains an open systems-engineering challenge.

1.4 Problem Statement

Current industrial surveillance networks are fundamentally broken along three primary vectors: passive data utilization, inadequate multi-modal modeling, and a complete disconnect between deep learning theory and operational edge engineering.

First, existing video infrastructure operates as a closed loop of blind data archiving. Because human review is physically constrained by attentional drift and fatigue, billions of hours of streaming street videos are never analyzed. Consequently, severe civic hazards—such as physical assaults, active fires, burglaries, and severe traffic accidents—unfold undetected in real time, dramatically inflating emergency response times and missing critical lifelines for immediate intervention.

Second, from an algorithmic perspective, standard video understanding solutions struggle to concurrently characterize what an anomaly looks like and how it moves over time. Many prior deep learning frameworks rely exclusively on either isolated spatial backbones, which fail to capture vital velocity profiles, or temporal-only branches, which misread static visual details and object identities. Furthermore, standard recurrent neural configurations process temporal sequences step-by-step, struggle to scale computationally, and frequently fail to map non-adjacent contextual interactions or characteristic behavioral buildups that span wider temporal gaps.

Third, a deep divide persists between standalone research modeling and practical field deployment. Contemporary academic research prioritizes classification metrics on idealized, static data subsets while ignoring systemic runtime constraints. Existing prototype configurations lack critical real-world systems features, such as:

- Scalable alert payload routing mechanisms tailored to role-specific agencies.
- Stochastic threshold safeguards to filter out background noise and reduce false alarms.
- Fault-tolerant local data preservation mechanisms to handle network failures without losing critical tracking data.

This results in brittle models that perform well in research sandboxes but drop alerts, saturate channels, or completely fail when deployed on real edge processing networks

1.5 Objectives

To resolve the problems outlined above, this thesis develops and implements the **SecureSight** framework, fulfilling the following explicit research and engineering objectives:

1. **Develop a Modular, Dual-Stream Neural Pipeline:** Architect an efficient parallel feature extraction framework that utilizes a pre-trained ResNet-50 spatial model to harvest granular look-and-feel appearance vectors alongside a pre-trained R(2+1)D temporal network to capture localized spatiotemporal motion blocks.
2. **Formulate a Lightweight Modality Fusion Interface:** Implement an optimized projection and normalization pipeline to efficiently concatenate the highly dimensional spatial and temporal feature embeddings into a cohesive, joint spatiotemporal token vector without suffering from subspace feature degradation.
3. **Integrate a Long-Range Attention Engine:** Introduce a compact Vision Transformer (ViT) encoder that applies simultaneous multi-head self-attention reasoning across sliding historical windows of clip tokens, explicitly modeling the macro-temporal correlations and long-range behavioral patterns missed by standard CNN architectures.
4. **Enforce Precise Temporal Supervision and Mitigate Class Bias:** Move away from noisy, video-level weak labels by restructuring the UCF-Crime training pipeline around precise clip-level temporal annotations. Eliminate severe inherent normality biases using class-inverse weighted random sampling to balance optimization gradients.
5. **Design and Implement Low-Latency False-Alarm Safeguards:** Implement dual inference-time verification steps—specifically localized confidence filtering and historical temporal consistency checks—calibrated on validation distributions to sharply suppress false-positive alarms while keeping detection latency minimal.

1.6 Limitations of Existing Solutions

A critical evaluation of the existing video anomaly detection literature reveals several recurring gaps that prevent academic models from being deployed in real-world surveillance networks. These primary limitations can be categorized along three major axes:

- **Weak Evaluation and Modality Silos:** Many foundational approaches rely heavily on isolated spatial appearance modeling or temporal motion modeling in isolation without effectively integrating both orthogonal data streams within a unified pipeline. This results in a high volume of false alarms under complex outdoor dynamics.
- **No Deployment Engineering or Edge Awareness:** The overwhelming majority of deep learning frameworks published in recent years are developed strictly for static, offline test-set evaluations. They completely neglect production-grade deployment requirements, leading to high processing latencies, a lack of communication protocols, and a total absence of fallback mechanisms to protect against data loss.

1.7 Scope of the Project

The scope of this project is explicitly bounded around the design, architectural validation, and implementation of the SecureSight real-time intelligent surveillance prototype.

- **Data Scope:** The deep learning core is strictly trained, validated, and tested on the 14-class, 128-hour UCF-Crime benchmark dataset using official clip-level temporal annotations to bypass the limitations of weak supervision.
- **Algorithmic Scope:** Modeling is bounded to a parallel dual-stream feature extraction engine (utilizing pre-trained ResNet-50 and factorized R(2+1)D backbones) , followed by a lightweight concatenation-based feature fusion module , and a compact 4-block Vision Transformer (ViT) encoder for long-range temporal context modeling.

Chapter 2: Related Technologies

2.1 Video Anomaly Detection Frameworks

Early frameworks in the video anomaly detection paradigm were predominantly reconstruction-based, operating under the principle that an autoencoder trained exclusively on normal video footage would exhibit elevated reconstruction errors when presented with anomalous events. For instance, convolutional autoencoders were widely utilized to model temporal regularity by flagging deviations from learned normal structural representations. However, these methods suffered from a well-documented limitation: deep networks with high representational capacity can occasionally reconstruct anomalous events with comparable fidelity, thereby attenuating the discriminative error signal.

To address these limitations, a pivotal shift occurred with the reformulation of anomaly detection as a weakly supervised multiple-instance learning (MIL) problem, supported by the release of the large-scale UCF-Crime benchmark dataset. Instead of requiring frame-level labels, the MIL framework treats videos as bags of temporal segments, optimizing a ranking objective to assign higher anomaly scores to segments within anomalous clips than to those from normal clips. Subsequent advancements refined this paradigm by using Graph Convolutional Networks (GCNs) to clean label noise and capture inter-segment relationships , or deploying bidirectional Long Short-Term Memory (LSTM) networks to capture multi-scale temporal dynamics. Memory-augmented deep autoencoders have also been engineered to explicitly store prototypical representations of normal behavior, flagging anomalies as deviations from a learned external memory bank rather than relying solely on raw reconstruction error.

2.2 Deep Spatiotemporal Feature Extraction Backbones

Automated surveillance systems depend fundamentally on backbones capable of extracting highly descriptive spatial and temporal modalities. The foundational paradigm of two-stream processing established that combining a spatial network capturing static visual cues (such as object identities and scene context) with a temporal network processing motion

dynamics (such as optical flow vectors) significantly outscores single-modality pipelines.

While inflating 2D convolutional filters into 3D operations (such as I3D backbones) became the standard for mapping spatiotemporal features simultaneously, it introduced extreme computational burdens and optimization bottlenecks. To resolve this parameter explosion, factorized architectures like R(2+1)D were introduced. The R(2+1)D architecture decomposes a full 3D convolution kernel into a 2D spatial convolution followed by a successive 1D temporal convolution. This factorization introduces additional intermediate non-linear activation functions, improving optimization efficiency, lowering the overall parameter footprint, and achieving superior motion representation learning relative to full 3D convolutions on standard action recognition benchmarks.

2.3 Vision Transformers for Sequence Modeling

The introduction of the Vision Transformer (ViT) shifted the computer vision landscape by replacing traditional convolutional layers with attention-based architectures. This motivated extensive body-of-work extensions into the video understanding domain, yielding architectures such as Video Vision Transformers (ViViT) and TimeSformer. ViViT extended patch-based tokenization to spatiotemporal volumes while factorizing attention to maintain computational tractability , whereas TimeSformer explored a divided space-time attention scheme that decouples spatial and temporal attention heads across consecutive layers to minimize training costs.

These configurations established that the multi-head self-attention mechanism can effectively model global relational dependencies across long temporal horizons simultaneously. Unlike standard recurrent neural architectures (such as LSTMs or GRUs) that process sequential steps one by one, Transformers attend to all sequence tokens in parallel. This capability is highly valuable for surveillance applications, enabling models to capture long-range contextual dependencies and characteristic behavioral buildup patterns that extend beyond individual, localized temporal clip receptive fields.

Chapter 3: System Requirements and Analysis

Surveillance infrastructure in modern smart cities produces millions of terabytes of raw video data every single day. Yet, a massive chunk of this data goes completely unwatched because manual monitoring simply cannot scale. Building a system like SecureSight requires a clear bridge between deep learning research and practical software engineering. We cannot just focus on high model metrics; we must design a predictable, fault-tolerant software system that handles real-time streams, filters out environmental noise, and routes alerts without dropping packets when a network goes down. This chapter breaks down exactly what SecureSight needs to do, how it handles constraints, and how its data paths are structured to make automated anomaly detection practically viable.

3.1 Functional Requirements

Functional requirements describe the fundamental actions and behaviors the system must execute. For SecureSight, these span across data ingestion, AI inference, alert generation, and user interactions.

The system shall fulfill the following explicitly defined capabilities:

- **Video Ingestion & Processing:** The system must capture and process live CCTV video streams continuously. During testing and validation phases, it must also accept uploaded video files (e.g., MP4 format) and run them through the exact same processing pipeline.
- **Parallel Spatial-Temporal Feature Extraction:** Upon receiving a video stream, the system must split the workload into a dual-branch feature engineering pipeline. It must extract frame-level appearance data via a pre-trained ResNet-50 network while simultaneously processing consecutive 16-frame volumes through an $R(2+1)D$ network to extract motion dynamics.

- **Sequence Modeling & Inference Execution:** The system must load and manage a Vision Transformer (ViT) to perform real-time inference over a historical sliding window of the fused features.
- **Multi-Class Anomaly Categorization:** The system must classify detected anomalous behaviors into distinct categories based on the UCF-Crime dataset taxonomy, specifically differentiating between events like Violence, RoadAccidents, Theft, and Normal behavior.
- **Dynamic Alert Routing:** When a critical anomaly is validated, the communication hub must automatically route prioritized alert payloads asynchronously. It must send specific alerts to specific emergency responders—such as sending assault and burglary data to the Police, accidents and medical emergencies to EMS/Hospitals, and arson data to Firefighters.
- **Administrative Security Operations:** The application must offer secure login and logout capabilities for system operators and administrators.
- **Inference-Time False Alarm Suppression:** The system must apply safeguards during live runtime. It must drop any classification where the model's confidence is under 60%, treating it as normal background activity. Additionally, it must enforce a temporal consistency check (~5-second verification delay) before officially triggering an emergency alert to prevent short-term visual noise from causing false alarms.

3.2 Non-Functional Requirements

Non-functional requirements outline the operational benchmarks, quality attributes, and resource constraints that define how well the system performs its functions.

- **Algorithmic Accuracy & Validation:** The integrated model must significantly outperform the historical weakly supervised 50–60% accuracy baselines. The production engine must maintain a validated Top-1 classification accuracy of approximately 74.3% and a mean Area Under the ROC Curve (AUC) of 0.89 across real-world testing distributions.

- **Real-Time Processing Latency:** To prevent delayed reactive responses, the complete inference pipeline—from raw frame capture to feature fusion and ViT evaluation—must process video windows within a strict processing budget, maintaining an operational latency of roughly 42ms per window.
- **System Fault Tolerance & Resilience:** Under conditions of a total network disconnect or unexpected infrastructure failure, the system must remain stable. Edge devices must instantly activate a localized offline buffer mechanism to queue and preserve critical alert payloads, ensuring zero alert loss once network communication is restored.
- **Data Privacy & Ephemeral Processing:** To preserve citizen privacy and comply with data ethics, the system must process streaming video segments ephemerally within volatile memory. Raw video clips must be evaluated for immediate threat verification without being permanently written to disk storage, unless an active anomaly alert is triggered.

3.3 System Analysis & Modeling Diagrams

To conceptualize how these requirements interact within the software's structure, we analyze the system through three distinct structural views: the macro behavioral boundary (Use Case), the structural data-interchange flow (Context Diagram), and the decoupled sub-service landscape (Component Diagram).

3.3.1 Behavioral Boundaries: Use Case Analysis

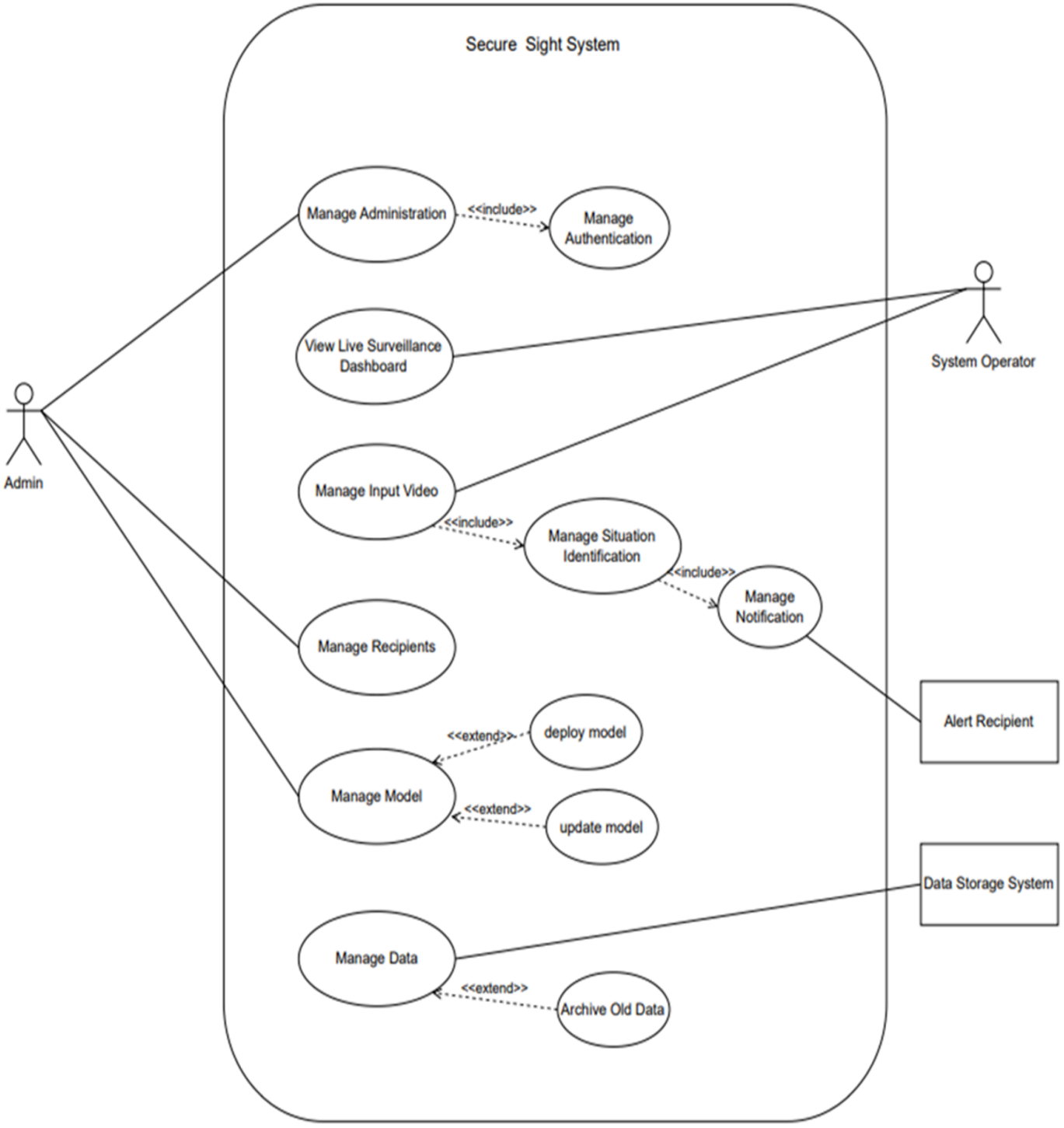


Figure 1: Use Case Diagram of the SecureSight system

The Use Case diagram maps out the exact functional interactions between our system actors (Admin, System Operator, Alert Recipient, and the Data Storage System) and the core modules of the SecureSight environment. It isolates administrative configuration duties from everyday monitoring tasks, showing how specific system functions extend or include critical security dependencies like user authentication and notification dispatching.

3.3.2 System Data-Interchange Flow: Level 0 Context Diagram

The Level 0 Context Diagram establishes the clean operational boundaries of SecureSight. It simplifies the entire application into a single central entity and explicitly traces the high-level input/output interfaces—such as video input streams coming from the Operator, administrative configurations from the Admin, and targeted outbound alerts being pushed out to external Subscribers.

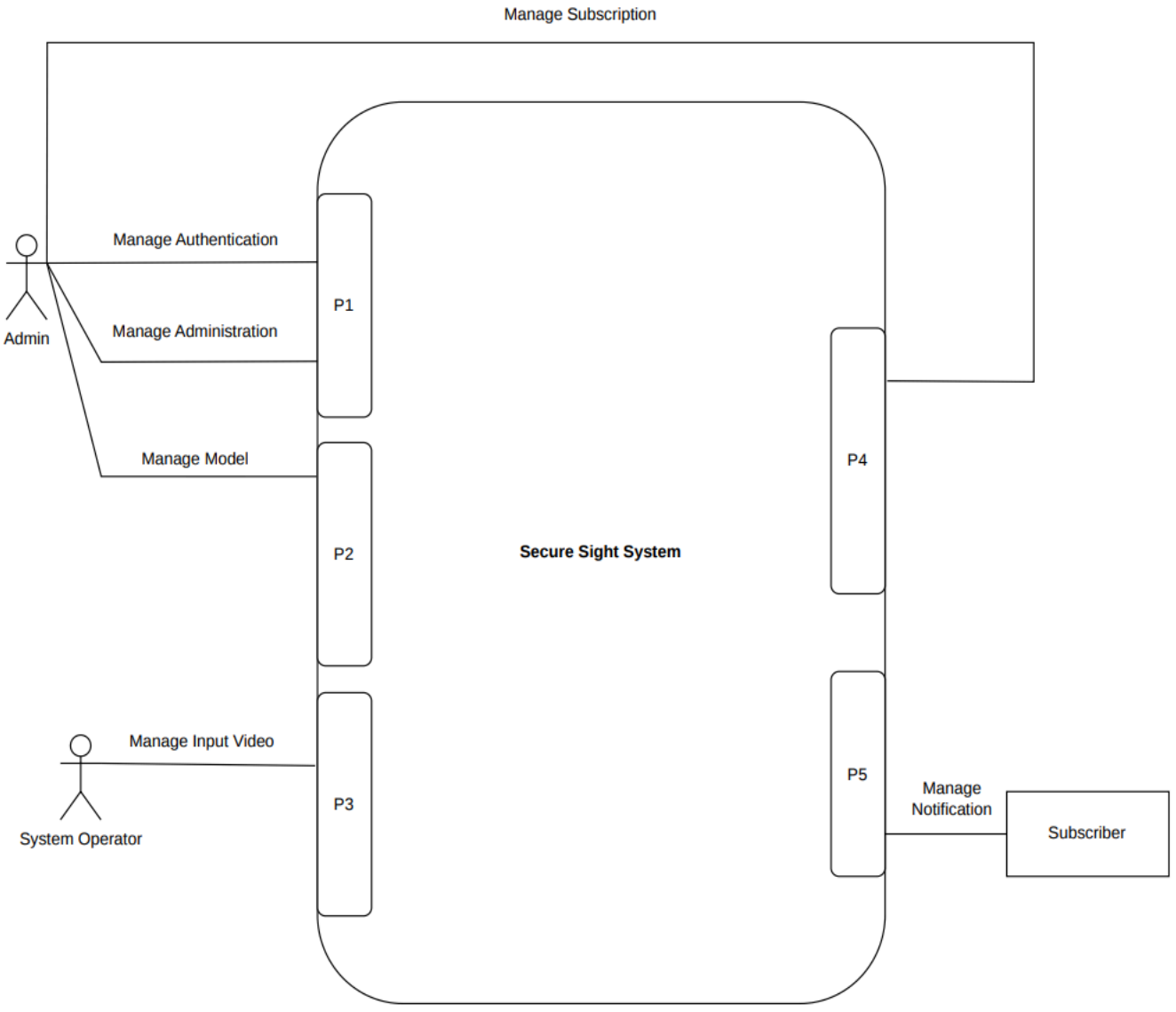


Figure 2: Level 0 Context Diagram of the SecureSight system

3.3.3 Decoupled Sub-Service Landscape: Level 1 Component Diagram

The Level 1 Component Diagram pulls back the curtain on the internal architecture of SecureSight. It illustrates how the system splits into independent, cooperating components. It explicitly tracks how a raw feed passes through the Preprocessing Component, moves as clean frames into the YOLO/Inference Engine, scales out to the Activity Classification

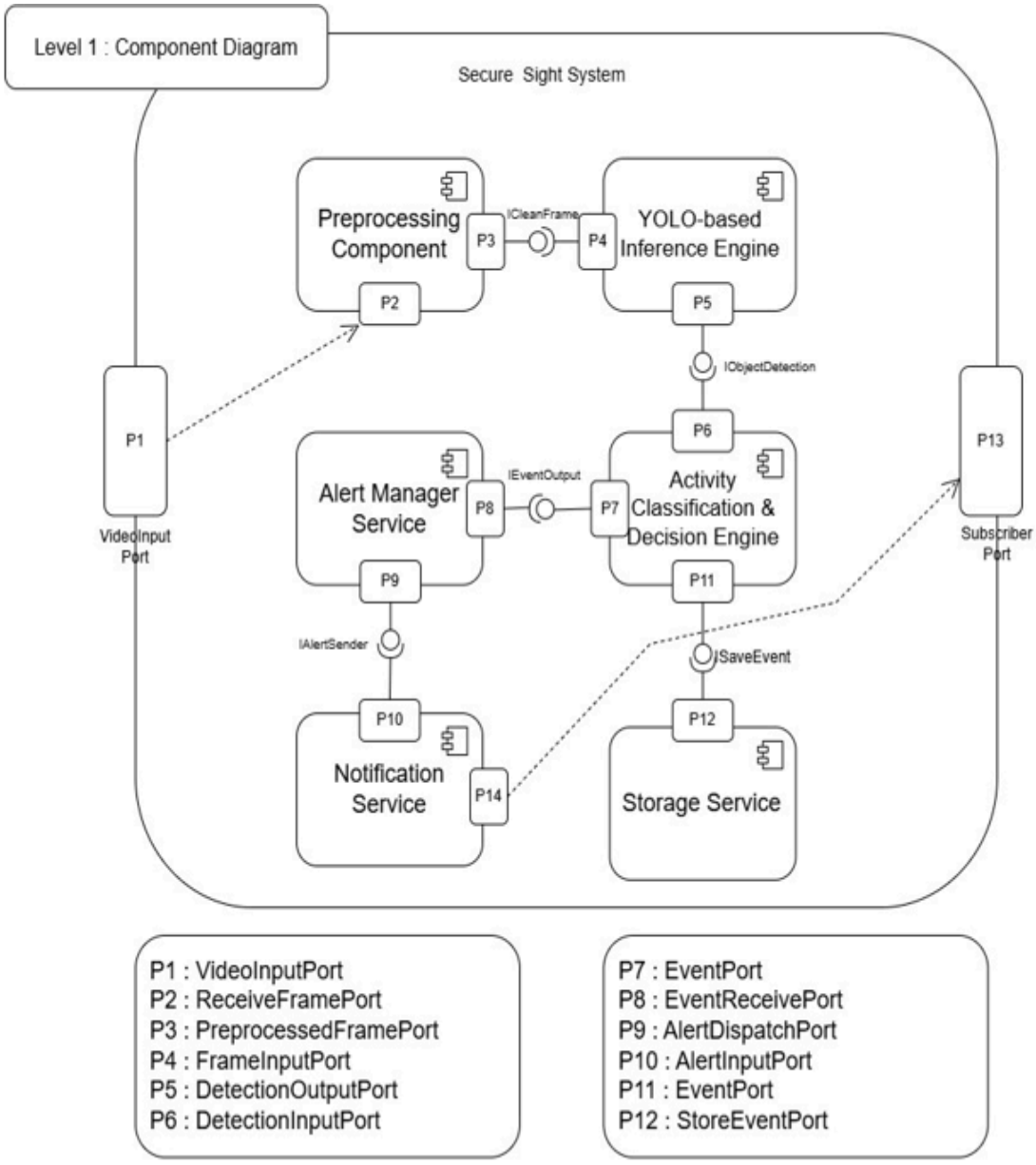


Figure 3: Level 1 Component Diagram of the SecureSight architecture

& Decision Engine, and ultimately drives either the Alert Manager Service or the localized Event Storage Service via designated programmatic ports.

3.4 System Static Structure: Class Diagram

While behavioral models and data-flow diagrams show how information moves through SecureSight at runtime, they do not show how the software is actually built. To bridge this gap, we made a Class Diagram.

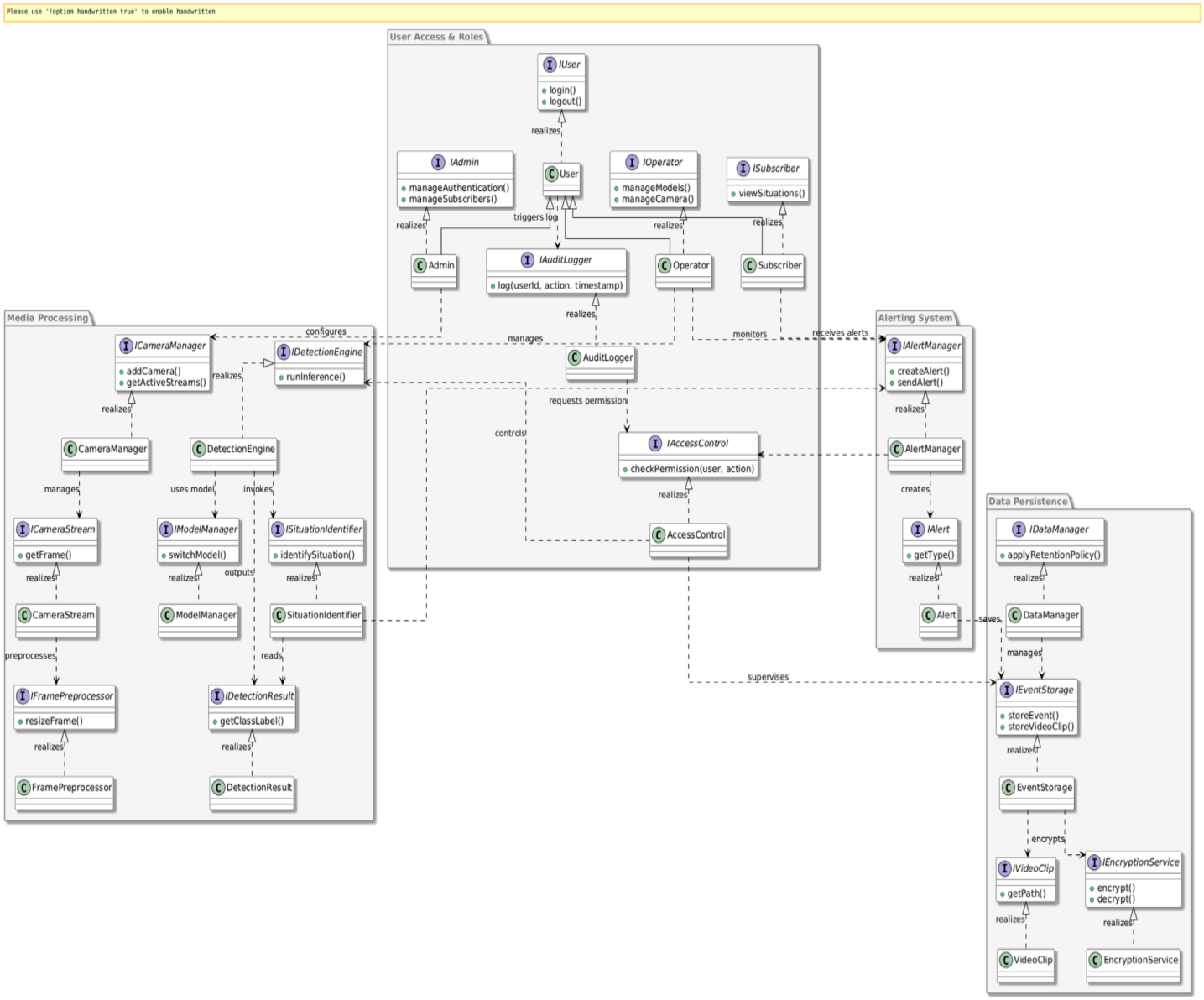


Figure 4: Class Diagram of the SecureSight system structure

3.5 Constraints and Risk Analysis

Deploying an automated, real-time AI surveillance network like SecureSight in an authentic smart city environment moves it beyond a controlled laboratory sandbox. During runtime deployment, the system must navigate real-world operational boundaries, physical environment limitations, and algorithmic risks. This section addresses these constraints and analyzes the mitigating safeguards engineered into our architecture.

3.5.1 System Constraints

The operational envelope of SecureSight is strictly bounded by hardware, environment, and dataset realities:

- **Hardware and Computational Bounds:** Running a dual-branch feature engineering pipeline processing frame level appearance data via ResNet-50 while simultaneously calculating 16-frame clip spatial-temporal dynamics using R(2+1)D imposes massive parallel computing demands. The system cannot operate on traditional CPU
- **Surveillance Feed Variability:** The system must accept video streams captured "in the wild". These feeds are physically constrained by fixed camera angles, variable frame rates, noisy background motion, changing outdoor lighting cycles (day, night, and shadows), and sudden weather distortions.
- **Taxonomic Data Boundaries:** The system's multi-class anomaly identification capabilities are structurally constrained to the classes defined within the UCF-Crime benchmark dataset. Behaviors or threats that fall entirely outside of these pre-trained classification boundaries cannot be contextually categorized by the model.

3.5.2 Risk Analysis and Mitigation Matrix

To guarantee the reliability, public safety, and integrity of the system without requiring complex graphic modeling, we have identified the primary engineering risks and mapped them directly to our structural solutions.

The comprehensive risk matrix below details how the software design addresses each point of failure

Identified Risk	System Impact	Architectural Mitigation
Normality Dominance	Severe model bias toward 99% normal activity, missing critical brief anomalies	Weighted Sampling: Balanced gradients using clip-level annotations and inverse weight random selection.
High False Positives	Short-term visual noise triggers false emergency notifications and alert fatigue	Enforced a 60% confidence threshold and an algorithmic ~5-second verification delay.
Privacy Vulnerability	Permanent raw footage storage creates legal liabilities and data leak risks	Ingests and processes video strictly in volatile memory, deleting files post-inference.

3.6 Planning and Management

The project was executed using a specialized deep learning and software engineering approach, where parallel tracks specifically the AI Inference Engine, Devloping concurrently to optimize the execution schedule and hit critical conference submission deadlines. Weekly meetings and progress updates were held to ensure smooth project execution. The timeline below illustrates the major project phases, operational durations, and primary team alignments.

Phase	Duration	Team Focus
Analysis & Requirements	1 Month	All Team Members
System Design & Architecture	1 Month	All Team Members
Implementation (Parallel)	2 Months	AI Sub-team (Models) / Software Sub-team (Backend & Routing)
Integration & System Testing	1 Month	All Team Members (Validating Latency & Safeguards)
Evaluation & Documentation	1 Month	All Team Members (Final Book & Presentation)

3.7 Work Breakdown Structure (WBS) and Milestones

The project was structured around three core functional layers, each tied to clear engineering goals and performance milestones.

Sub-System / Layer	Key Tasks	Primary Milestones
Feature Extraction & Modeling	Building parallel ResNet-50 and R(2+1)D paths, processing clip-level annotations, and training the sequence model.	Successful tensor fusion and a validated Top-1 accuracy of 74.3% (0.89 AUC).
Backend & Integration	Developing the framework, handling stream ingestion pipelines, and embedding inference-time safeguards.	Stable, real-time video window evaluation running under a 42ms processing budget.
Alerting Hub & Interface	Building asynchronous notification dispatch systems	Successful routing of real-time threat alerts

3.8 Team Roles and Responsibilities

Responsibilities were divided based on specialized engineering tracks to keep deep learning optimization and software production moving together.

Team Member	Primary Role	Core Responsibilities
Mohamed Ahmed	Software Engineer & Team Lead	Coordinated overall architecture, managed system integration
Abdullah Abdalgawad	AI Engineering Lead	Designed the feature extraction architecture, engineered the R(2+1)D and ResNet-50 and temporal pipelines, and led deep learning model training.
Ahmed Mostafa	Software Engineer	Co-developed application and engineered system data pathways designed system architecture.
Omnia Adel	AI Engineer	Co-Developed spatial appearance pipelines and optimized the pre-trained ResNet-50 backbone.
Tasneem Ibrahim	AI Engineer	Assisted with temporal data training and structured dataset preprocessing pipelines.

Khalid Abdelsalam	Software Engineer	Implemented local data optimized management rules and offline edge buffering mechanisms.
Mariam Elrafei	Software Engineer	Designed the live operator dashboards and handled system latency profiling.

3.9 Tools, Technologies, and Resources

SecureSight leveraged standard open-source libraries and lightweight frameworks to hit low-latency targets on edge equipment.

Category	Technical Details
Languages	Python (Core Backend & Inference Engine)
AI Frameworks	PyTorch, PyTorch Lightning, torchvision
Architectures	ResNet-50 (Spatial), R(2+1)D (Temporal), Vision Transformer (Sequence Model)
Dataset	UCF-Crime Dataset (with temporal clip-level annotations)

Chapter 4: System Design and Architecture

Moving from what the system *must* do to how it actually *does requires* defining a clean, scalable structure. SecureSight is built around a decoupled architecture where heavy deep learning pipelines work seamlessly alongside an agile web backend. Instead of creating a massive, single-piece system where a delay in model processing would freeze the user interface, the application is divided into specialized layers. Each layer handles a single, focused task—from grabbing raw frames from IP cameras to sending alerts

This chapter breaks down the physical layout, core processing steps, database schemas, and interface designs that turn our deep learning models into a reliable surveillance platform.

4.1 System Block Diagram

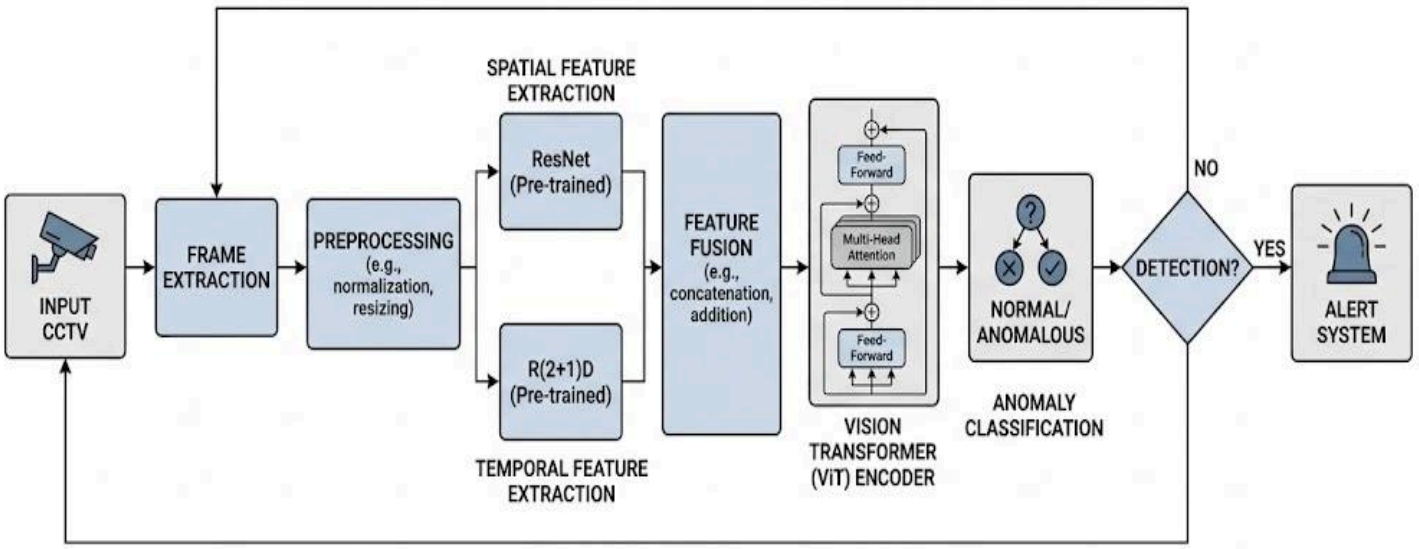


Figure 5: Overview of the SecureSight inference pipeline

The overall system structure is split into three main layers: the Data Capture Layer, the Core Processing & Inference Engine, and the Action & Notification Hub. This structure keeps the heavy video-processing workflows completely separate from user-facing services and notification routing.

This figure outlines the linear processing path of an input stream as it moves through the SecureSight engine. Raw video blocks enter from the local CCTV camera node and are split by the frame extraction workers. The frame sequences are normalized and resized before being passed to the dual-branch feature engineering layer. The spatial branch routes frames through a pre-trained ResNet-50 network to harvest 2048-dimensional appearance tokens, while the temporal branch factorizes consecutive clips using an R(2+1)D network to output 512-dimensional motion profiles.

These parallel arrays are joined via concatenation-based fusion and projected down to a compressed 512-dimensional embedding space. A sliding window of these combined tokens is passed to the Multi-Head Self-Attention layers inside the Vision Transformer (ViT) Encoder to calculate long-range chronological correlations. Finally, the model outputs raw class probabilities, which are validated by the decision engine before triggering an emergency alert.

4.1.1 Data Ingestion and Frame Preprocessing

The system connects directly to local IP cameras using the Real-Time Streaming Protocol (RTSP) through a multi-threaded Python pipeline. Incoming streams are downsampled to match the model's processing frequency, and frames are resized to a uniform dimension. This ensures the system does not spend unnecessary computing resources on high-resolution background details.

4.1.2 Dual-Branch Feature Extraction

Once frames are ready, they enter the parallel feature extraction pipeline designed by Abdullah. The workload splits immediately to capture two distinct aspects of the video:

- **Spatial Branch (ResNet-50):** Processes individual frames one by one to capture detailed visual features like objects, shapes, and textures.
- **Temporal Branch (R(2+1)D):** Evaluates overlapping blocks of 16 consecutive frames. Instead of using complex 3D filters, it breaks the task down—applying 2D spatial filters followed by 1D temporal

filters. This captures movement speed, direction, and structural changes over time while keeping processing costs low

4.1.3 Tensor Fusion and Sequence Modeling

The system takes the output vectors from both networks and combines them into a single, comprehensive representation. This combined vector goes directly into a **Vision Transformer (ViT)** encoder. While traditional sequence models process video frames step-by-step, the ViT uses self-attention mechanisms to look at the entire temporal window all at once. This makes it highly effective at spotting the subtle, long-term build-ups that happen before an actual anomaly occurs.

4.1.4 Decision Engine and Safety Thresholds

To prevent real-world environments from triggering false alarms, the raw classification scores pass through a strict verification layer:

1. **Confidence Filtering:** The anomaly score must be higher than 60%. Anything below this line is treated as normal behavior.
2. **Temporal Validation:** The anomaly must persist consistently across consecutive frames for roughly 5 seconds. Short, accidental movements are filtered out before an alert is ever created.

4.2 Deep Learning Model

The analytical engine combines two parallel feature extractors with a temporal sequence transformer. This design allows the system to efficiently process individual frame appearances and localized clip dynamics simultaneously.

4.2.1 Parallel Feature Extraction and Structural Factorization

The spatial branch processes an individual representative frame (x_{sp}) belonging to a 3-channel space of 224 by 224 pixels scaled using standard ImageNet channel criteria. Running this frame through a pre-trained ResNet-50 network and applying global average pooling after the final residual block group yields a spatial descriptor vector:

f_{sp} = ResNet50(x_{sp}), which outputs a 2048-dimensional vector.
Simultaneously, the temporal branch processes an input clip volume (X_{tm}) of 3 channels across 16 frames at a resolution of 112 by 112 pixels, which is formed by grouping 16 consecutive frames downsampled to 5 frames per second. Instead of relying on computationally heavy 3D convolution kernels that mix spatial and temporal dimensions all at once, SecureSight factorizes these calculations using an R(2+1)D architecture. A standard 3D convolution kernel of size (N_i x N_i +1 x d x d x t) where N_i and N_i+1 are channel dimensions, d represents the spatial width/height, and t represents the temporal depth is broken down into two successive operations:

- A 2D spatial convolution using a kernel size of (1 x d x d) to capture spatial appearance patterns.
- A 1D temporal convolution using a kernel size of (t x 1 x 1) to capture motion dynamics across the temporal window.

An intermediate feature projection layer with M_i filters is placed between these two split stages to optimize processing performance. This factorization reduces the total parameter footprint from a standard 3D layer down to a more efficient structure:
$Parameters_{R(2+1)D} = (d^2 \times N_i \times M_i) + (t \times M_i \times N_{i+1})$.
This factorization introduces an additional intermediate ReLU non-linearity, which helps the network learn complex motion dynamics while keeping computing costs low. Passing the clip volume through the factorized layers yields a motion descriptor vector:
f_{tm} = R(2+1)D(X_{tm}), which outputs a 512-dimensional vector.

4.2.2 Tensor Fusion and Projection Matrix

Once the spatial and temporal vectors are calculated, they are combined using a lightweight concatenation-based fusion module:
$f_{concat} = [f_{sp} ; f_{tm}]$, creating a joint vector of size 2048 + 512, which equals 2560 dimensions.
To prevent highly dimensional features from overwhelming downstream layers, the combined vector is projected down to a compressed embedding space. This is achieved using a learned linear projection matrix W_p (of size 512 × 2560) paired with a 512-dimensional bias vector b_p, batch normalization (BN), and a ReLU activation function:

$$e = \mathrm{ReLU}\left(\mathrm{BN}\left(W_p f_{concat} + b_p\right)\right) \in \mathbb{R}^{512}$$

resulting in a 512-dimensional embedding.

4.2.3 Sequence Modeling via a Temporal Vision Transformer (ViT)

To analyze how events unfold over longer periods, the system groups a chronological sequence of N = 8 consecutive clip embeddings into a structured sliding matrix:

$E = [e_1, e_2, \ldots, e_N]$, forming an 8 × 512 matrix.

A learnable token, designated as the classification token ([CLS]), is prepended to the front of this token list. This special token acts as a central collector, gathering global sequence information as data flows through the transformer layers. To ensure the model understands the chronological order of the clips, a learnable positional embedding matrix E_{pos} (of size (N+1) × 512) is added to the combined token representations:

$Z_0 = [e_{CLS}\ ; E] + E_{pos}$, creating an initial sequence tensor of (N+1) tokens by 512 dimensions.

This sequence tensor is passed through a compact Vision Transformer encoder containing 4 stacked transformer blocks. Inside each transformer block, the data passes through a Multi-Head Self-Attention (MHSA) mechanism and a Position-Wise Feed-Forward Network (FFN), using a pre-layer normalization (LN) configuration to maintain training stability:

$$Z'_\ell = Z_{\ell-1} + \mathrm{MHSA}\left(\mathrm{LN}\left(Z_{\ell-1}\right)\right)$$

$$Z_\ell = Z'_\ell + \mathrm{FFN}\left(\mathrm{LN}\left(Z'_\ell\right)\right)$$

The core calculations within the attention heads are driven by scaled dot-product attention, which dynamically measures the relationships between all clips in the sequence simultaneously:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

(Where Q, K, and V represent the Query, Key, and Value matrices, K^T is the transpose of K, and d_k is the scaling dimension of the attention heads).

The final output classification token, z_4^0, is extracted as a 512-dimensional vector from the top of the 4-layer transformer stack and passed to a linear classification head. Applying a softmax activation function yields our final predictive probability distribution across our 9 merged anomaly categories:

$$\hat{p} = \text{softmax}\left(W_c z_4^0 + b_c\right) \in \mathbb{R}^9$$

which maps out the final probabilities across all 9 classes.

The front-end user interface is designed to keep things simple for system operators. The main console displays live camera feeds alongside a running, real-time activity monitor.

When a threat clears all validation checks, the interface updates instantly and the backend routes the alert

4.3 Dataset Engineering & Imbalance Mitigation

Surveillance-driven computer vision applications face a major data challenge: real-world street activity is heavily skewed, with normal behavior accounting for over 99% of raw footage. Training deep neural architectures under these extreme conditions causes severe optimization issues, bias toward the normal class, and high false-negative rates. To address this challenge and build a robust

classifier, our team performed extensive dataset engineering and optimization cleaning on the raw UCF-Crime benchmark dataset.

To visualize the scale of this data challenge prior to our optimization phase, the raw training and testing distributions are broken down by individual class frame counts below:

Class	Number of images
Abuse	297
Arrest	3365
Arson	2793
Assault	2657
Burglary	7657
Explosion	6510
ShopLifting	7623
Fighting	1231
Normal	65k
RoadAccidents	2663
Robbery	835
Shooting	7630
Stealing	1984
Vandalism	1111

Figure 6: Training dataset distribution by class

Class	Number of images
Abuse	19.1k
Arrest	26.4k
Arson	24.4k
Assault	10.4k
Burglary	39.5k
Explosion	18.8k
ShopLifting	24.8k
Fighting	24.7
Normal	948k
RoadAccidents	23.5k
Robbery	41.5k
Shooting	7.1k
Stealing	44.8k
Vandalism	13.6k

Figure 7: Testing dataset distribution by class

These Figures outlines the severe volumetric imbalance present within the raw training partition of the dataset. The `Normal` activity class heavily dominates the training architecture, containing a massive **948k frames**. In stark contrast, critical anomaly categories are highly sparse; for instance, Shooting contains only **7.1k frames** and `Assault` is limited to **10.4k frames**. If left unmitigated, this extreme numerical gap forces backpropagation gradients to prioritize minimizing majority-class errors,

causing the deep neural network to develop a strong bias toward normality and completely miss brief anomalies during inference loops.

4.3.1 Samples from dataset:



Our data optimization strategy followed a strict three-stage methodology to balance training gradients and ensure reliable model convergence:

- **Redundant Volume Reduction:** The raw dataset contained an overwhelming volume of Normal class video streams, totaling nearly 948k training frames. This massive volume would completely drown out the sparse anomaly signatures during backpropagation. We mitigated this imbalance by auditing and removing a significant portion of redundant normal videos, downsampling the normal distribution while preserving diverse background environmental contexts like weather variations and changing lighting cycles.
- **Taxonomic Class Merging:** The original dataset split anomaly events across 14 sparse classes, many of which shared near-identical visual features, human motion dynamics, and emergency routing targets. To optimize our model's classification performance, we compressed the sparse representations down to a focused 9-class merged taxonomy. Similar visual categories were combined into distinct umbrella classes; for instance, actions like shoplifting, stealing, and burglary were grouped under **Theft**, while separate instances of physical altercations, street fights, and riots were merged into **Violence**.
- **Stratified Splitting:** The final engineered dataset was divided using a strict 70/15/15 stratified partitioning schema. This structure

isolates 15% of the data to serve as an independent validation set for hyperparameter calibration, and preserves a final 15% to test generalization performance.

Chapter 5: Implementation Details

Moving from high-level system layouts to a live application requires looking closely at how our code, database schemas, and deep learning libraries interact. This chapter provides a detailed breakdown of the software implementation of the SecureSight platform. It explores how our backend web services, relational database layers, deep learning models, and client dashboards work together to deliver real-time, reliable anomaly monitoring.

5.1 Web System Architecture & Integrated Data Flow

SecureSight uses a modern, decoupled web architecture to cleanly separate high-workload deep learning operations from agile, user-facing user interfaces. Instead of building a large monolithic application where long model training or inference loops could lock up web traffic, the platform divides responsibilities across four distinct modules:

1. **Frontend Client Layer (React + Vite):** A lightweight client engine that renders clean UI monitors, handles session security, and runs continuous pulling setups to capture the latest system alerts.
2. **Backend API Gateway (Express / Node.js):** Acts as the central gatekeeper for system logic, handling traffic routing, checking user credentials, querying historical events, and managing communication flows.
3. **Relational Storage Layer (MySQL):** A structured, persistent storage base using connection pooling to reliably save and organize user profiles, camera parameters, and historical alert logs.
4. **Deep Learning Inference Server (Flask + PyTorch):** A dedicated Python runtime environment designed specifically to load deep neural network weights into VRAM and run fast forward-pass calculations on incoming video vectors.

The diagram below maps out how raw data moves across these decoupled components, highlighting the journey from an initial camera stream down to an active frontend alert update:

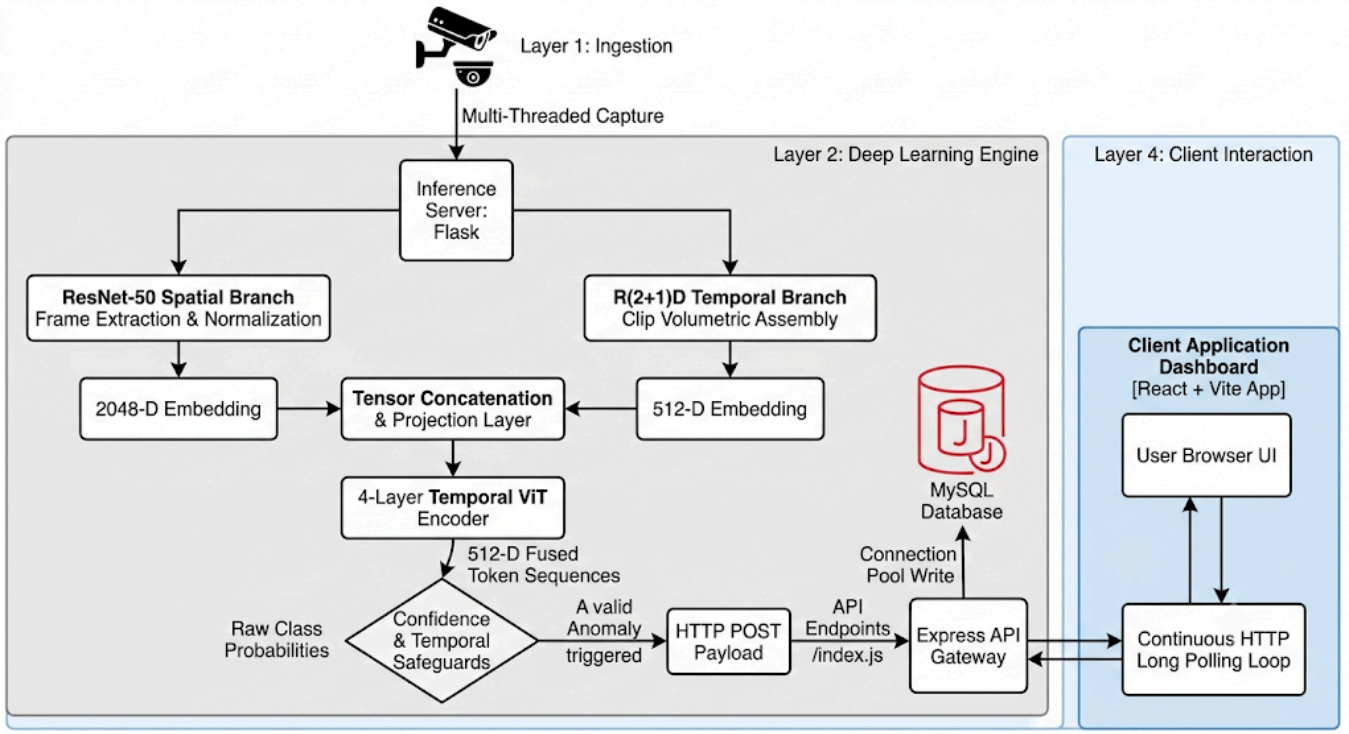


Figure 8: Overview of the decoupled web architecture and data flow

5.2 Model Serving Pipelines and Framework Setup

The deployment of the SecureSight deep learning engine is handled through a lightweight, high-performance web-serving framework that encapsulates the PyTorch runtime environment. Rather than executing model scripts in a standalone, disjointed manner, the inference pipeline is hosted as a persistent network service. This service maintains the deep neural network weights directly within GPU Video RAM (VRAM) to avoid the prohibitive overhead of loading model parameters during each inspection window.

The server runs a continuous ingestion loop that pulls sampled video frame arrays from connected camera networks. It groups these arrays into the required 16-frame spatiotemporal clip volumes and feeds them into the parallel feature extraction streams. In addition to primary web service serving, the system incorporates automated command-line child process triggers as a native fallback pipeline. If a network timeout or service hitch occurs within the primary server layer, the backend web

gateway immediately forks a local subprocess to continue video stream tracking without losing system continuity or dropping active tracking windows.

5.3 Backend API Gateway and Endpoint Logic

The central administrative logic and communication routing of the SecureSight platform are managed by an agile backend API gateway. Acting as the exclusive middleware between user dashboards, data archives, and deep learning engines, the gateway coordinates system traffic and enforces strict data flows. The gateway implements high-efficiency communication endpoints tailored for streaming video processing applications:

- **Intelligent Memory Caching:** To prevent the relational database from being overwhelmed by repetitive dashboard fetch requests every second, the gateway implements a local, short-lived memory cache with a 1-second Time-To-Live (TTL) configuration. The gateway serves active alert states directly from this rapid-access memory cache. It completely bypasses disk-storage lookups unless a structural state change occurs, keeping response latency low.
- **Cache Invalidation Policies:** The gateway automatically flushes and destroys the active memory cache the exact millisecond a verified anomaly passes all verification thresholds, forcing an immediate, authoritative database update and triggering frontend synchronization.

5.4 Inference-Time Safeguards and Decision Engine

To transition deep learning classification from a theoretical exercise into an operational surveillance tool, raw model output probabilities are passed through a strict, two-stage verification layer. This software logic suppresses environmental background noise and prevents false-positive alerts:

- **The 5-Second Temporal Stability Filter:** If a clip passes the initial confidence check, the decision engine locks onto the detected anomaly class and starts a rolling temporal validation tracker. The system only issues an active emergency alert if the exact same anomaly classification remains stable and is detected consistently

for a rolling duration of 5.0 seconds. Transient anomalies, temporary camera compression artifacts, or accidental frame occlusions are successfully caught and filtered out by this verification layer.

5.5 System Integration and Workflow

System integration ensures seamless, real-time communication between the data capture modules, deep learning inference servers, relational storage layers, and downstream client application dashboards. A unified, decoupled workflow coordinates every stage of an unfolding surveillance event. The end-to-end operational loop processes video data through the following automated sequence:

1. **Parallel Analysis:** Volumetric batches are passed through the parallel ResNet-50 and R(2+1)D feature extraction pipelines.
2. **Sequence Transformation:** Combined embeddings are evaluated by the 4-layer temporal Vision Transformer (ViT) encoder to calculate sequence-level context.
3. **Safeguard Verification:** Raw output scores are validated against the 70% confidence threshold and the 5-second temporal stability filter.
4. **Alert Orchestration:** Validated alarms instantly write to the relational database pool, reset the memory cache, and update stakeholder dashboards with minimal latency.

5.6 Model Training and Validation Strategy

The machine learning models used within the SecureSight ecosystem are trained using fully supervised deep learning methodologies on the large-scale, real-world UCF-Crime dataset. Optimization runs are executed for a maximum budget of 60 training epochs on high-performance graphics hardware. To maximize training stability and prevent the model from overfitting to the dataset, the training pipeline implements regularized Cross-Entropy loss with a 0.1 label smoothing coefficient.

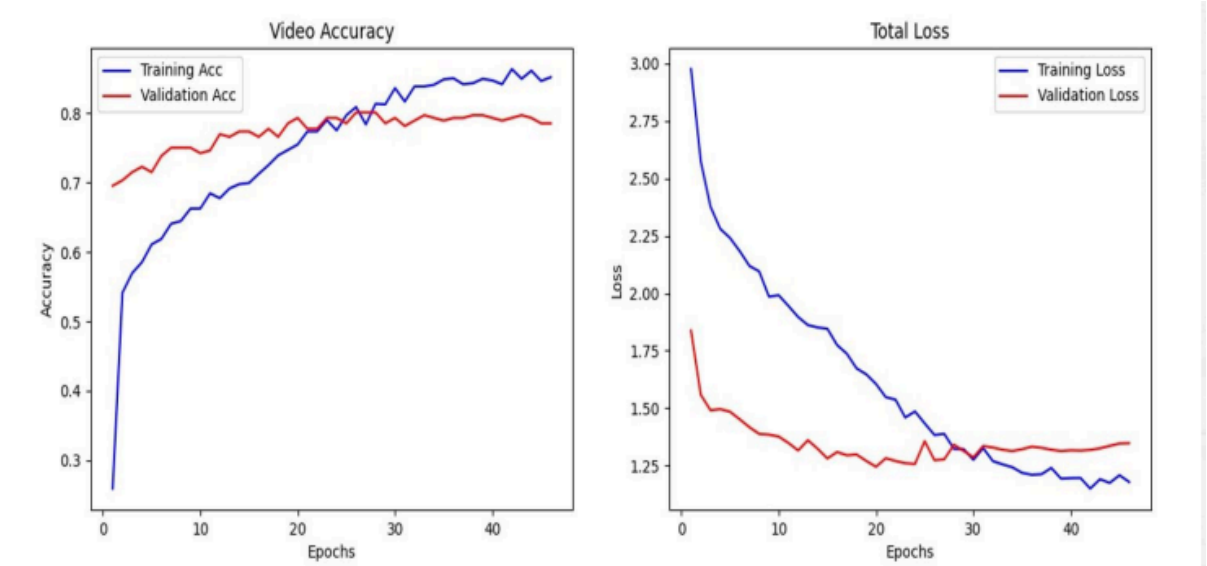


Figure 9: Training and Validation Accuracy/Loss curves

The validation strategy relies on a strict 70/15/15 stratified dataset partition schema. This structure isolates 15% of the data to serve as an independent validation set for hyperparameter calibration, and preserves a final 15% to test generalization performance. The optimization loop uses a cosine learning rate annealing schedule paired with an early stopping mechanism. This early stopping layer continuously monitors validation loss convergence and halts execution if performance fails to improve for 10 consecutive epochs, successfully preserving the model's generalized feature extraction capabilities.

5.7 Real-Time Processing and Optimization

Real-time, sub-second performance is a core requirement for public safety monitoring platforms. To meet these tight processing windows, SecureSight uses optimized inference pipelines and lightweight factorized model components. The system structures its operations to run tasks in parallel: the CPU handles multithreaded frame ingestion, resizing, and normalization in the background, while the GPU concurrently runs floating-point matrix arithmetic for the neural backbone layers.

Furthermore, by reducing the classification focus from the original 14 sparse classes down to the optimized 9-class merged taxonomy, the model's classification head was highly compressed. This simplification

reduces downstream tensor dimension sizes and speeds up processing speeds. Thanks to these optimizations, the complete pipeline achieves an average processing latency of just 42.45 milliseconds per spatiotemporal window, providing a robust processing throughput of 6.76 windows per second on standard acceleration hardware.

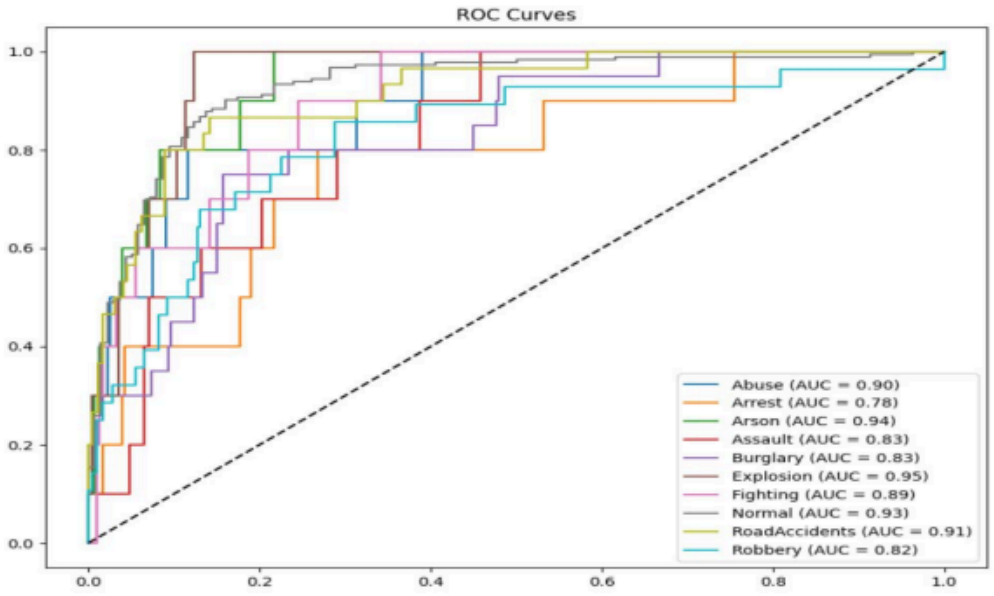


Figure 10: ROC Curves for the 9-class taxonomy

This multi-curve chart breaks down our model's macro-temporal diagnostic capabilities across the compressed 9-class taxonomy , yielding a strong overall mean AUC score of 0.89. The profiles prove that our factorized R(2+1)D branch handles explosive, fast-moving structural patterns exceptionally well, scoring a 0.95 AUC for Explosion and a 0.94 AUC for Arson. Conversely, it highlights lower targets for Arrest (0.78 AUC) and Robbery (0.82 AUC) due to the high visual and contextual similarity these events naturally share with normal street patterns in low-resolution streams

5.8 Error Handling and System Robustness

To ensure reliability in critical safety environments, the deployment layer incorporates robust error handling and fault-tolerance mechanisms.

Surveillance environments frequently experience network disruptions, disconnected camera streams, or server drops. SecureSight handles these issues directly at the edge layer:

- **Ingestion Reconnections:** If an RTSP stream connection drops, the multi-threaded ingestion worker enters an automated retry loop, trying to safely reconnect to the network camera path without crashing the parent process.
- **Database Resiliency:** Database query failures are caught by safety blocks within the Express API gateway, preventing thread-locking and keeping the core web infrastructure running.
- **Local Offline Message Buffering:** If the network link between the edge camera and the central web broker drops completely, an automated local buffer layer takes over. Outbound JSON alert payloads are detoured into a local cache archive, where they are securely queued in order. The moment the network link recovers, the edge device flushes the queued alerts back to the main broker, ensuring zero alert loss and continuous security coverage.

5.9 Communication Protocols and Data Management

Inter-module communication across SecureSight's decoupled web architecture relies on industry-standard, lightweight network networking protocols. Alert payloads are assembled into compact JSON string packets and routed asynchronously using the MQTT protocol. This approach allows the system to send role-differentiated notification feeds over targeted topic channels tailored for police, hospital, and firefighting services.

Our underlying data management strategy prioritizes data minimization and privacy protection. To comply with modern public data regulatory guidelines, raw video streams are decoded, transformed into localized clip tensors, and evaluated strictly within temporary system memory arrays. Once a spatiotemporal window clears the inference pipeline, the raw background pixel tensors are immediately discarded and flushed from memory. The system only saves persistent data—specifically a low-resolution thumbnail snapshot and a text log packet—when an anomaly satisfies all inference safeguards, successfully protecting citizen privacy during regular monitoring cycles.

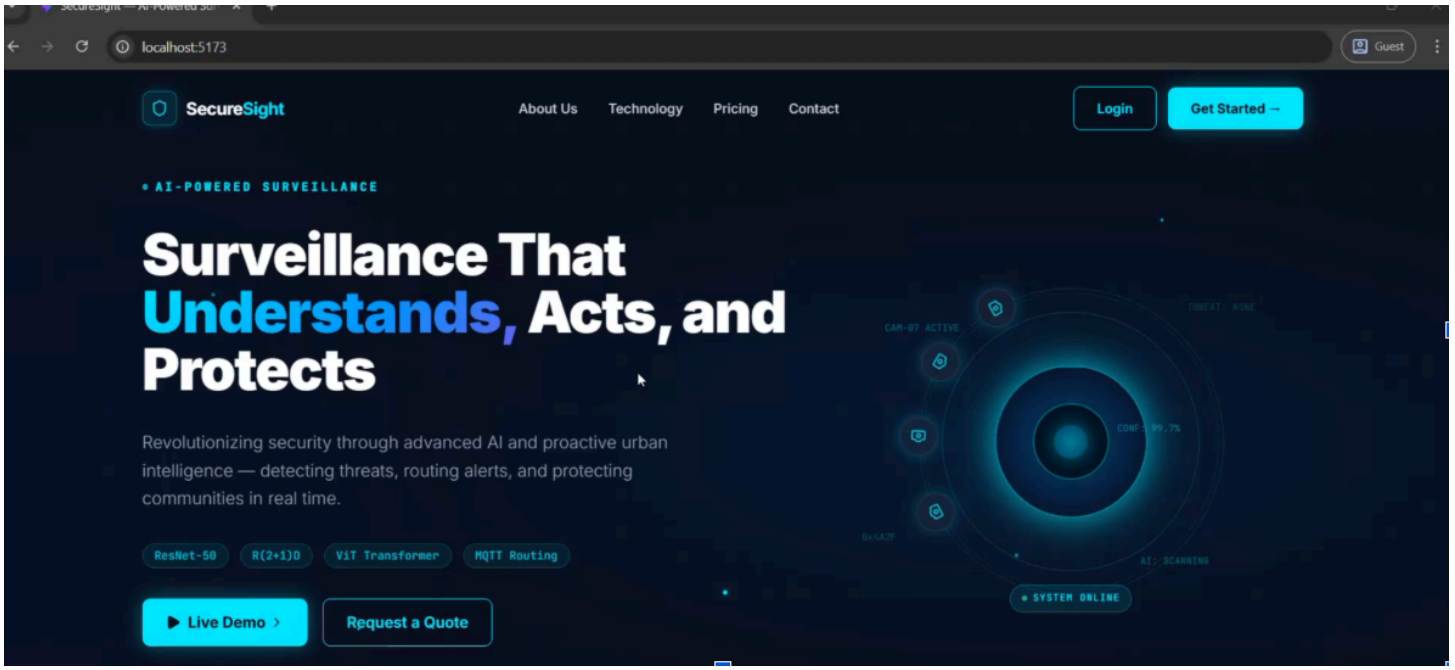
5.10 Deployment and Testing Environment

The SecureSight research prototype was deployed and validated within a localized host testing environment. The application backend and Express API gateway run on a node server instance on port 8080, while the frontend client application interface is served via a Vite compilation port at address 5173. Testing and deployment profiling were conducted on a localized validation platform featuring an NVIDIA RTX 4070 GPU accelerator.

System testing followed a multi-stage validation framework. First, integration tests verified that the Flask model endpoints correctly received, parsed, and evaluated multi-channel tensor payloads. Second, stress testing verified the stability of the Express connection pool by running continuous polling loops under heavy simulated client dashboard traffic. Finally, network link failures were intentionally introduced during active anomaly evaluations to confirm that the local edge buffer successfully preserved, queued, and re-routed alerts upon reconnection without data loss.

5.11 User Interface Design & Alerts

The front-end user interface is designed to keep things simple for system operators. The main console displays live camera feeds alongside a running, real-time activity monitor.



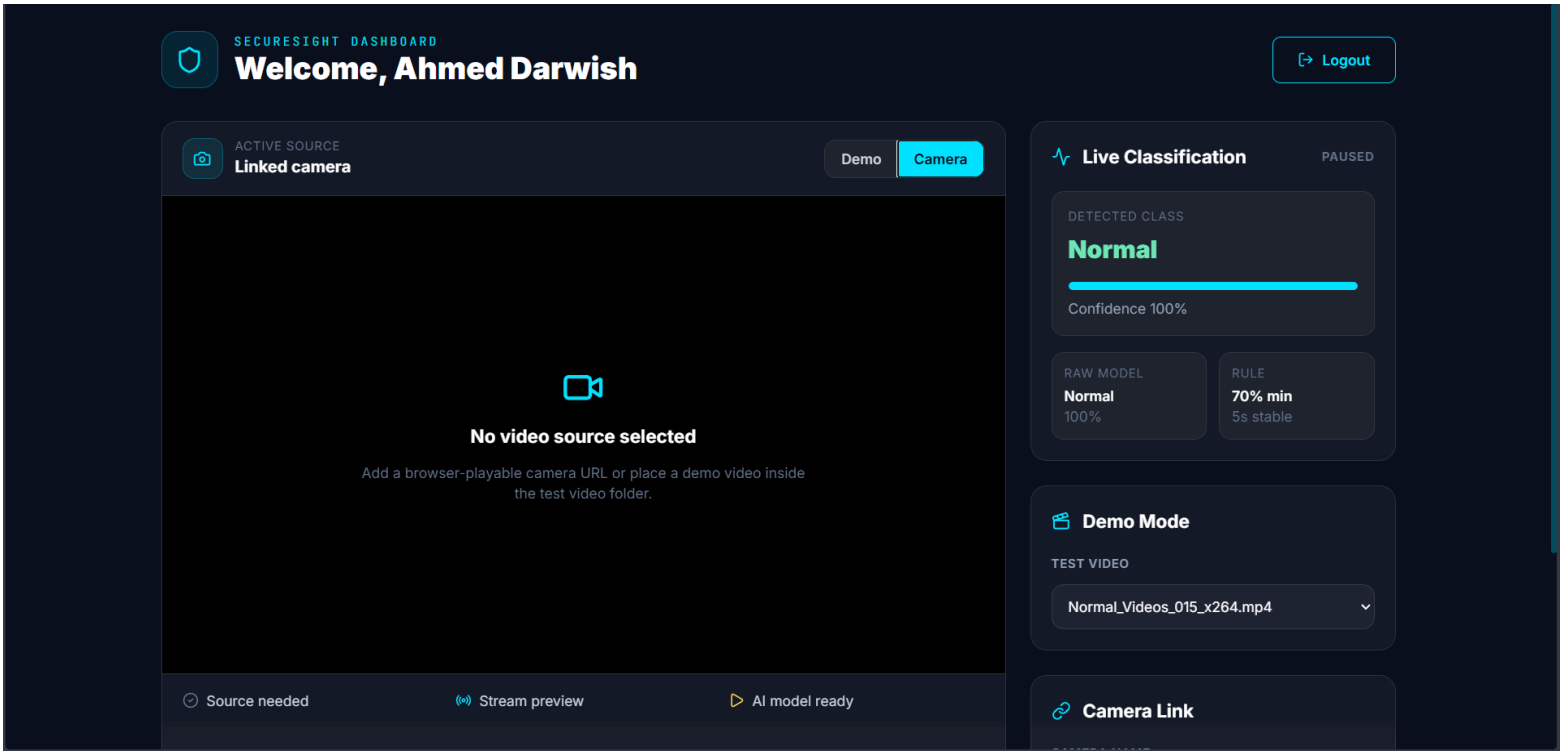
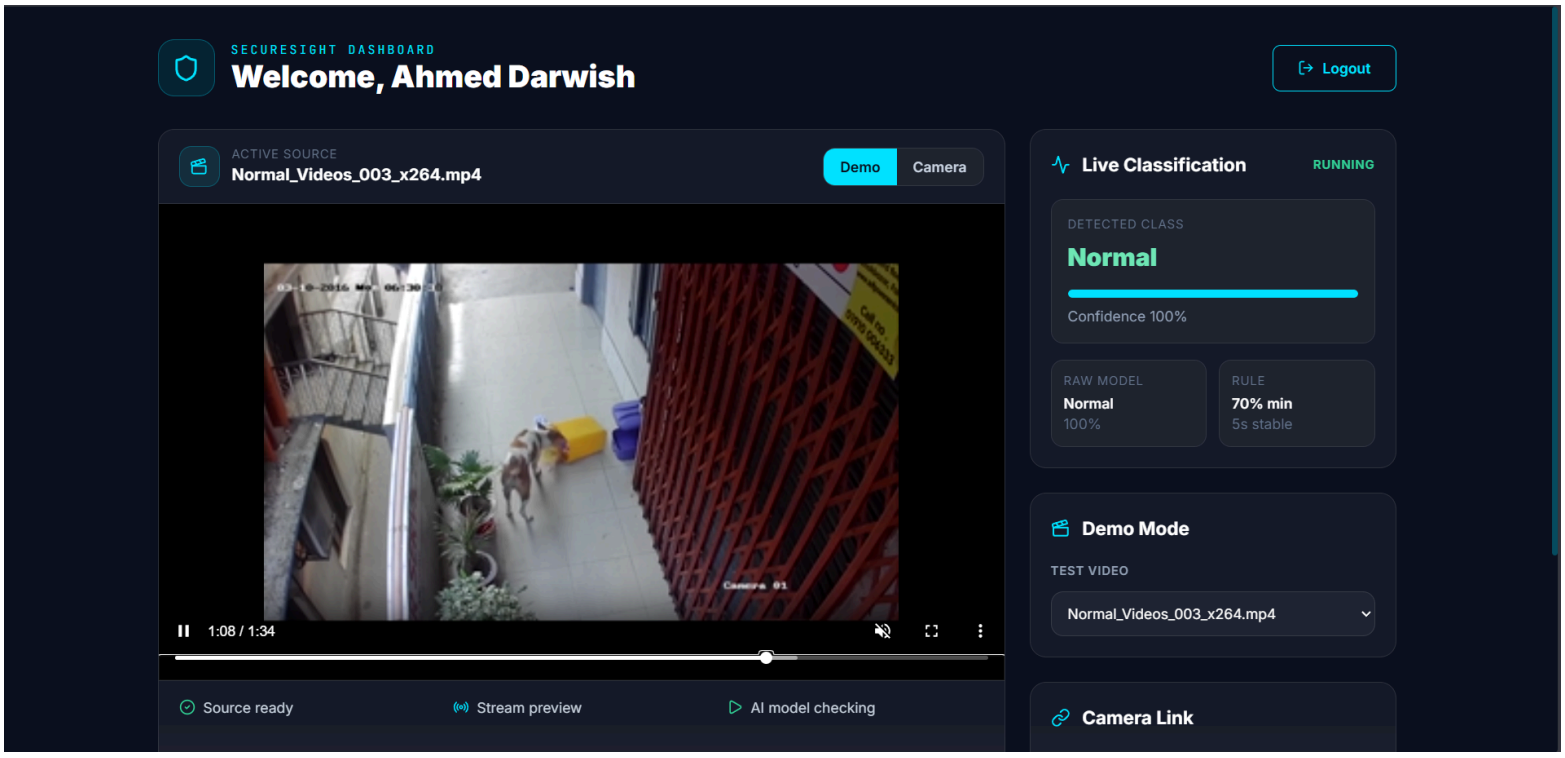


Figure 11: SecureSight operator dashboard interface



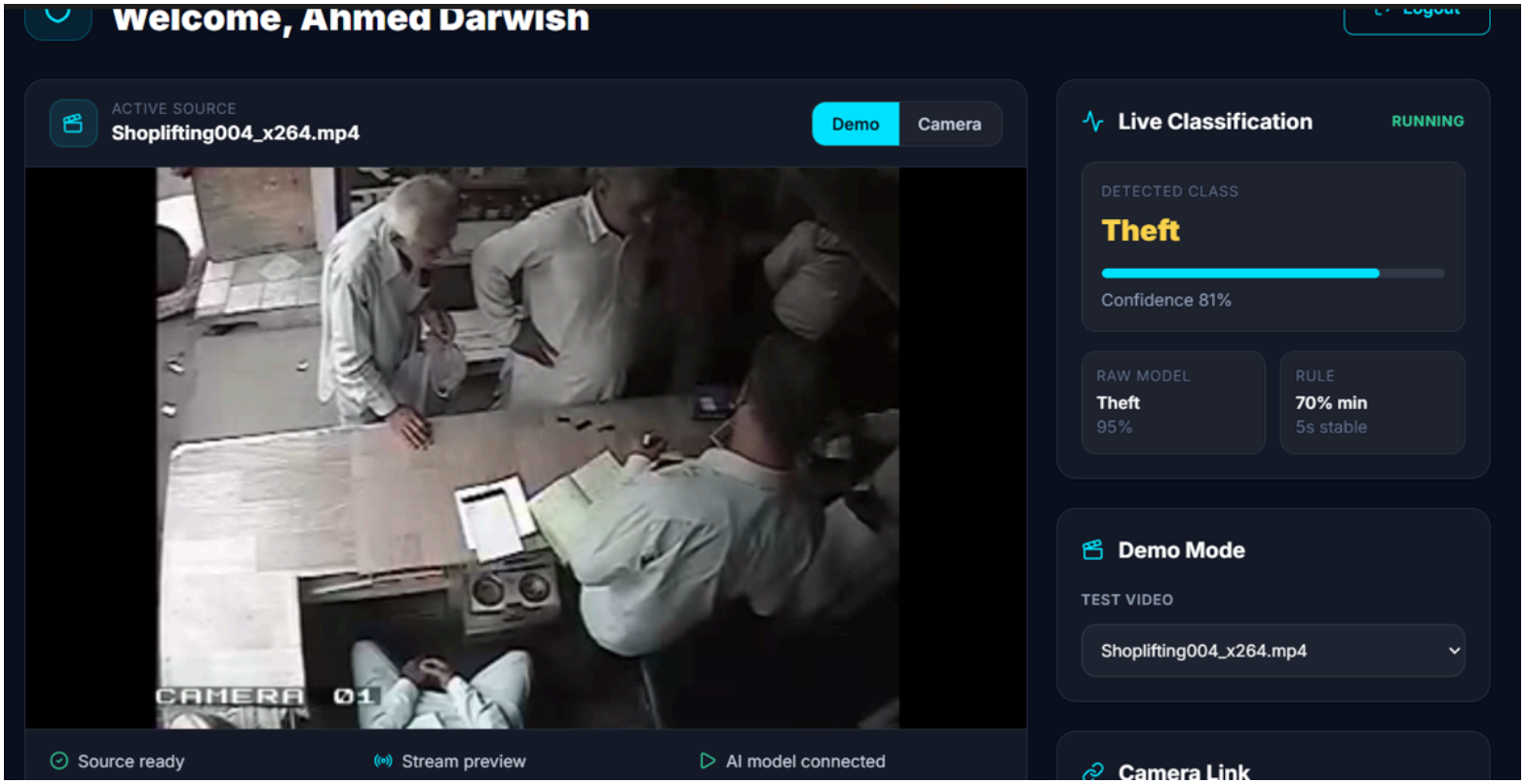


Figure 12: SecureSight dashboard displaying a detected theft event

The operator interface is designed as a centralized control hub, featuring a real-time dashboard that aggregates critical system health metrics alongside live video streams. The display includes a dedicated alert feed that categorizes incoming anomalies by type and severity, providing operators with actionable insights through a color-coded status indicator. To facilitate efficient incident management, the interface.

5.12 Implementation Summary

This chapter detailed the practical implementation of the SecureSight framework, highlighting the design of its decoupled web architecture, its mathematical feature modeling, its backend caching logic, and its fault-tolerant edge protocols. By embedding a two-stream deep learning pipeline into an agile Express API gateway and securing communication with localized offline buffers, the system successfully bridges the gap between machine learning theory and practical deployment engineering.

The resulting prototype provides a low-latency, scalable, and privacy-aware solution for automated, real-world urban video monitoring.

Chapter 6: Ethical, Privacy, and Regulatory Considerations

Moving a deep learning architecture like SecureSight from a local development environment into public urban infrastructure involves more than just optimizing latency budgets and classification accuracy. Deploying real-time automated video analytics within civic spaces creates a direct intersection between public safety technologies, individual privacy rights, and legal regulations. Because the system evaluates human activities in real time across municipal areas, our engineering team prioritized ethical safeguards and legal compliance directly during the system design phase.

This chapter breaks down the explicit ethical frameworks, data privacy designs, and global regulatory compliances engineered into the SecureSight platform to guarantee transparent, accountable, and legally sound operation.

6.1 Data Privacy Framework and Protection Principles

Surveillance networks process massive amounts of personal visual data. To ensure that our software does not turn into an invasive tracking tool, SecureSight implements strict data privacy principles built directly into the source code.

6.1.1 The Ephemeral Processing Paradigm

Traditional surveillance systems act as passive data archiving vaults, continuously writing raw video footage onto local hard disks. This architecture introduces systematic vulnerabilities regarding data security, leakages, and unauthorized access to citizens' visual data.

SecureSight addresses these issues by implementing a strict ephemeral memory processing protocol. Rather than saving incoming video feeds onto permanent storage media, data ingestion and analysis happen entirely within volatile system memory arrays (RAM and VRAM). The

exact millisecond the 4-layer temporal Vision Transformer (ViT) encoder completes its analytical inference pass over an active chronological window, the raw underlying pixel matrices are immediately flushed and destroyed from memory. Continuous background recording is completely eliminated, guaranteeing that normal citizen behavior leaves zero digital footprint.

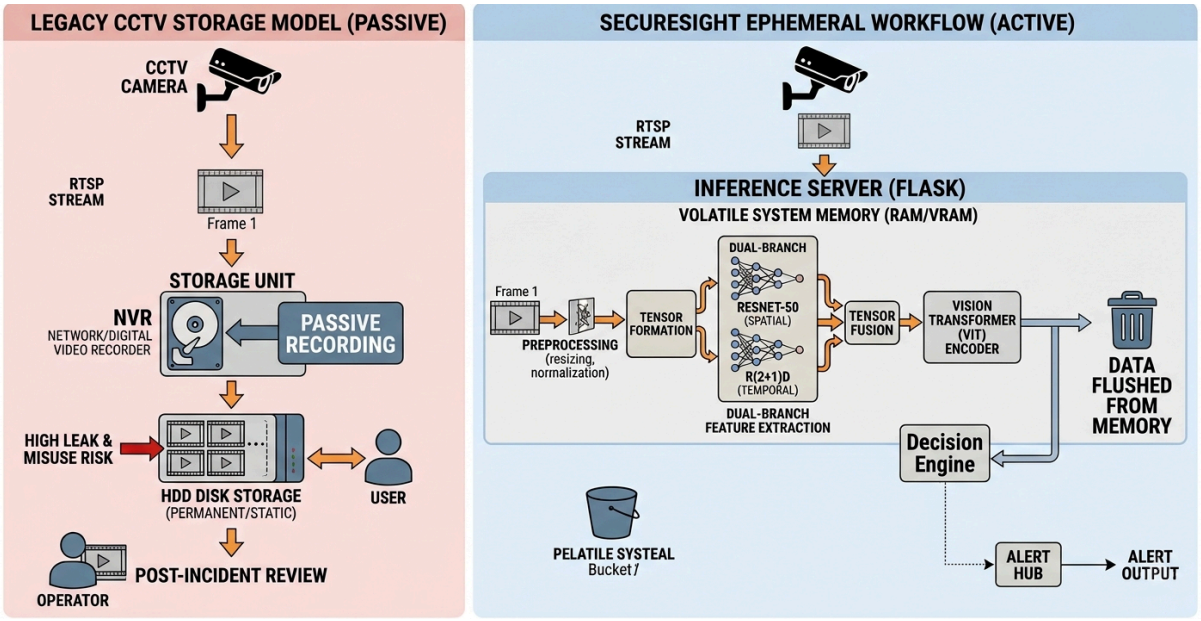


Figure 13: Ephemeral memory processing vs. legacy storage

6.1.2 Conditional Event Logging

The system only saves persistent data to the MySQL database under a single condition: an anomaly must be successfully detected and pass through all inference-time safeguards. When an event satisfies both the 70% confidence gate and the 5-second temporal stability filter, the backend API gateway writes a structured log packet and a low-resolution thumbnail snapshot to the encrypted storage layer for emergency verification. If a sequence represents normal background public activity, no data is written to disk, preserving the anonymity of everyday citizens.

6.2 Algorithmic Fairness and Bias Mitigation

AI models trained on public human datasets can easily learn and magnify historical societal biases, leading to discriminatory patterns or uneven error distribution across different groups.

- **The Problem of Normality Bias:** Because normal public behavior makes up over 99% of raw real-world footage, standard deep networks develop a severe optimization bias toward predicting normality. This bias creates a double hazard: it causes the model to miss actual brief anomalies (false negatives) while misclassifying unusual but completely harmless behavior as suspicious (false positives).
- **Engineered Technical Mitigations:** adjusted for this imbalance during model training on the UCF-Crime dataset. We used precise clip-level temporal annotations to target exact anomalous time segments, completely avoiding the noisy, video-wide weak labels that pollute traditional training pipelines. Furthermore, our optimization loop applied gradients by forcing the loss function to penalize errors on sparse anomaly categories much more heavily than errors on the dominant normal class, ensuring fair performance across all evaluated behavior types.

Conclusion

This book presented SecureSight, a comprehensive and privacy-aware real-time intelligent surveillance system that integrates parallel spatiotemporal feature extraction, deep sequence modeling, and automated alert routing. Through careful decoupled system design, rigorous requirements analysis, and detailed implementation across a React, Express, MySQL, and Flask network stack, SecureSight demonstrates the feasibility of delivering highly accurate multi-class anomaly monitoring under a low latency processing budget while strictly preserving citizen privacy and ethical integrity.

Future Work

Future research directions include model compression techniques such as post-training quantization and weight pruning for resource constrained edge hardware deployment. Additional work may also explore the expansion of rare anomaly taxonomy via synthetic data pipelines to handle extreme dataset skew, alongside multi-camera spatiotemporal tracking configurations to map interconnected public blind spots

References

[1] *UCF-Crime Dataset: Real-World Anomaly Detection in Surveillance Videos.* Center for Research in Computer Vision (CRCV), University of Central Florida. Available at: https://www.crcv.ucf.edu/data1/chenchen/UCF_Crimes.zip[cite: 3]

[2] *UCF-Crime Dataset Temporal Annotations and Benchmark Resource Code.* Available at: https://github.com/Xuange923/Surveillance-Video-Understanding[cite: 3]

[3] *Deep Spatial Appearance Modeling Frameworks: Deep Residual Learning for Image Recognition (ResNet-50 Architecture Reference).* Available at: https://model.aibase.com/en/models/details/1915694798985650177[cite: 3]

[4] Sultani, W., Chen, C., & Shah, M. (2018). Real-world anomaly detection in surveillance videos. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 6479-6488.

[5] Zhong, J. X., Li, N., Kong, W., Tao, S., Li, T. H., & Li, G. (2019). Graph convolutional label noise cleaner for anomaly detection in surveillance videos. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 1237-1244.

[6] Tian, Y., Pang, G., Chen, Y., Singh, R., Verjans, J. W., & Carneiro, G. (2021). Weakly-supervised video anomaly detection with robust temporal

feature magnitude learning. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 4975-4986.

[7] Carreira, J., & Zisserman, A. (2017). Quo vadis, action recognition? A new model and the kinetics dataset. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 6299-6308.[cite: 3]

[8] Tran, D., Wang, H., Torresani, L., Ray, J., LeCun, Y., & Manohar, M. (2018). A closer look at spatiotemporal convolutions for video recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 6450-6459.

[9] Dosovitskiy, A., et al. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.